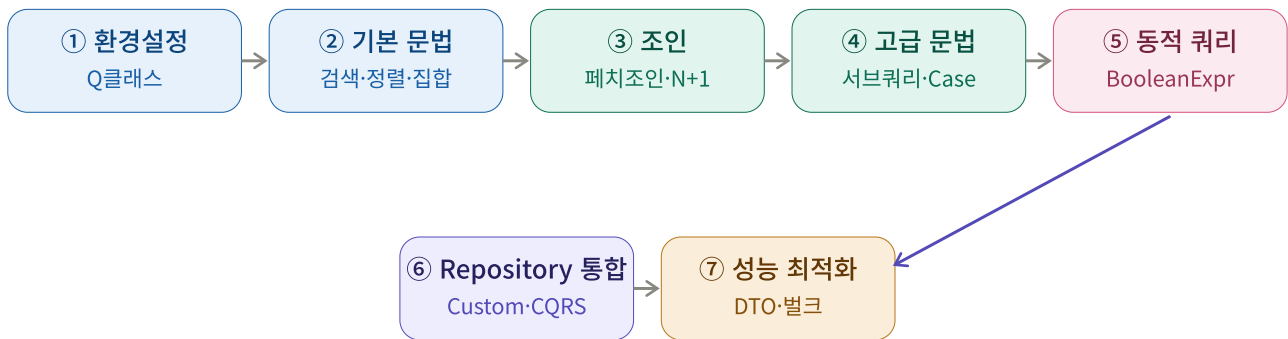


[Lv2] JPA 5주차 학습 노트

실무의 꽃, QueryDSL로 동적 쿼리와 성능 최적화 정복

문자열 JPQL의 오타는 런타임(장애)에서야 터진다. QueryDSL은 쿼리를 **자바 코드**로 짜서 컴파일 시점에 문법 오류를 100% 차단한다. if 문으로 짜깁기하던 동적 쿼리를 '레고 조립'하듯 리팩토링하고, 수십 줄을 메서드 한 줄로 압축한다.



문자열의 불안함을 없애고 자바 코드로 쿼리를 지배한다

▲ 5주차 전체 흐름 (⑧ SQL 함수 호출까지)

① 환경 설정과 Q클래스의 비밀

JPQL은 문자열이라 오타(`usernadm`)를 컴파일러가 못 잡고 **런타임에 장애**가 난다. QueryDSL은 자바 코드라 오타에 빨간 줄이 그어지고 IDE 자동완성이 완벽 지원된다.

```
// build.gradle - Spring Boot 3.x (Jakarta) 전용
implementation 'com.querydsl:querydsl-jpa:5.0.0:jakarta'
annotationProcessor 'com.querydsl:querydsl-apt:5.0.0:jakarta'
annotationProcessor 'jakarta.annotation:jakarta.annotation-api'
annotationProcessor 'jakarta.persistence:jakarta.persistence-api'
```



Q클래스 = 빌드 결과물 → git 커밋 금지 (/build 는 .gitignore)

▲ Q클래스는 엔티티의 메타데이터(거울) — 쿼리 조립용 경로 객체를 제공

```
@Configuration
public class QuerydslConfig {
    @PersistenceContext private EntityManager em;
    @Bean public JPAQueryFactory jpaQueryFactory() { return new JPAQueryFactory(em); }
}
```

```
// static import로 짧게: import static ...QuerydslMember.querydslMember;
QuerydslMember m = queryFactory.select(querydslMember).from(querydslMember)
    .where(querydslMember.username.eq("hello")).fetchOne();
```

빨간 줄이 뜬다면 Cannot resolve QMember → build 폴더 우클릭 → [Mark Directory as] → [Generated Sources Root]. 필드를 추가했으면 compileJava 로 Q클래스를 **재생성**해야 한다.

② 기본 문법 — 검색·정렬·페이징·집합

SQL 연산자와 거의 1:1로 매칭되므로 자동완성에 의존하면 된다. **AND는 콤마(,)로 나열**하는 게 가독성도 좋고 동적 쿼리에서 null을 무시해주는 이점이 있다.

```
queryFactory.selectFrom(querydslMember)
    .where(
        querydslMember.username.contains("team"), // LIKE '%team%'
        querydslMember.age.between(10, 30),        // BETWEEN
        querydslMember.age.goe(20)                  // >= 20 (콤마 = AND)
    ).fetch();
// 연산자: eq(=) ne(!=) in between goe(>=) gt(>) loe(<=) lt(<) like startsWith contains
```

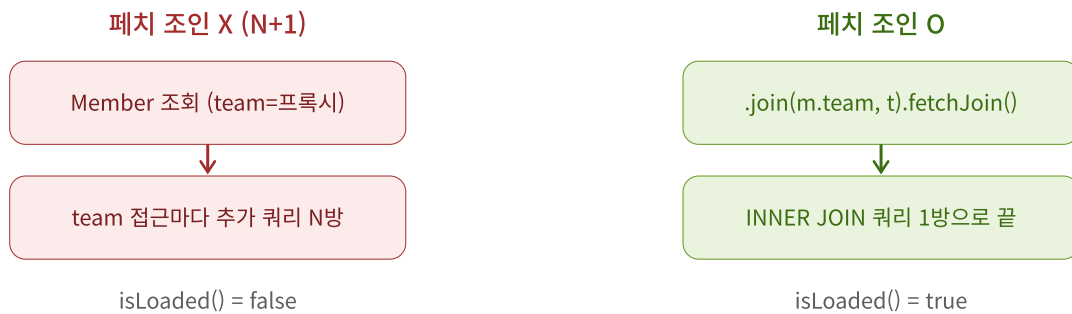
메서드	설명	리턴
<code>fetch()</code>	리스트 조회. 없으면 빈 리스트	List<T>
<code>fetchOne()</code>	단건. 없으면 null, 2개 ↑면 예외	T
<code>fetchFirst()</code>	<code>limit(1).fetchOne()</code>	T
<code>fetchResults/fetchCount</code>	Deprecated (5.0+): GROUP BY 카운트 버그	—

페이징·집합 카운트는 `select(member.count()).fetchOne()` 으로 **분리**하는 게 표준. 정렬은 `.desc().asc().nullsLast()` , 페이징은 `.offset().limit()` . 집합함수(count/sum/avg/max)는 **Tuple**로 반환되며 **Repository 안에서만** 쓰고 Service로 던지지 않는다(QueryDSL 종속 타입).

③ 조인과 페치 조인 — N+1 해결사

`join(경로, 별칭)` 형태. 첫째는 조인 대상 경로, 둘째는 Q타입 별칭이다.

조인	문법	설명
기본(inner)	<code>.join(member.team, team)</code>	연관관계 따라 조인
세타	<code>.from(member, team)</code>	연관관계 없이 where로 매칭
ON 필터	<code>.leftJoin(...).on(...)</code>	외부조인 대상 필터링
페치	<code>.join(...).fetchJoin()</code>	N+1 해결



▲ JPQL의 문자열 "join fetch" 대신 메서드 한 방 — 실수할 일이 없다

```
QuerydslMember m = queryFactory
    .selectFrom(querydslMember)
    .join(querydslMember.team, querydslTeam).fetchJoin() // ✨ 핵심
    .where(querydslMember.username.eq("member1"))
    .fetchOne();
// emf.getPersistenceUnitUtil().isLoading(m.getTeam()) == true
```

④ 서브 쿼리와 고급 문법

서브 쿼리는 `JPAExpressions` 를 쓴다. `queryFactory` 가 `EntityManager`를 물고 있는 **실행용**이라면, `JPAExpressions` 는 `EntityManager` 없는 **조립용 부품**이다(WHERE·SELECT만 가능, FROM 절 불가).

```
QQuerydslMember sub = new QQuerydslMember("memberSub"); // 서브쿼리 별칭 필수
queryFactory.selectFrom(querydslMember)
    .where(querydslMember.age.eq(
        JPAExpressions.select(sub.age.max()).from(sub) // 나이 최대인 회원
    )).fetchOne();

// Case (웬만하면 자바에서 처리)
new CaseBuilder().when(member.age.between(0,20)).then("학생").otherwise("어르신")
// 상수·문자더하기 (숫자는 stringValue() 필수)
member.username.concat("_").concat(member.age.stringValue()) // "member1_10"
```

⑤ 동적 쿼리 — BooleanBuilder vs Where 다중 파라미터

사용자가 "이름만" 또는 "나이만" 검색할지 모르는 상황. JPQL의 if문 지옥을 QueryDSL이 천국으로 만든다.

① BooleanBuilder

```
if(name != null)
    builder.and(...)
if(age != null)
    builder.and(...)
→ 단순할 때만
```

② Where 다중 파라미터 (실무 99%)

```
.where(usernameEq(name),
        ageEq(age))
조건이 null 반환하면
QueryDSL이 자동 무시
→ 재사용·조립 가능
```

▲ Where 다중 파라미터: null 반환 = 조건 자동 삭제

```
// 반환 타입은 Predicate가 아닌 BooleanExpression! (.and/.or 조립 가능)
private BooleanExpression usernameEq(String c) {
    return c != null ? querydslMember.username.eq(c) : null; // null이면 무시됨
}
// 조립(Composition): 비즈니스 조건을 재사용
private BooleanExpression isAdPossible(Integer age, String name) {
    BooleanExpression a = ageGoe(age), n = usernameEq(name);
    if (a == null) return n; if (n == null) return a; // null 방지
    return a.and(n); // "광고 가능 회원"이라는 의미가 코드에 드러남
}
```

✅ **조립의 장점 재사용**(ageGoe는 어디서든), **가독성**(`where(isAdPossible(...))` 만 봐도 의도 파악), **유지보수**(조건 바뀌면 메서드 하나만 수정 → 모든 쿼리 자동 반영).

⑥ Repository 통합 — Custom Repository "3단 합체"

QueryDSL을 Service에 직접 쓰면 서비스가 "비즈니스 로직"이 아닌 "쿼리 조립"으로 도배된다. 3단 구조로 **JpaRepository의 편리함 + QueryDSL의 강력함**을 하나로 합친다.



Spring이 이름 규칙(Impl)으로 구현체를 자동 연결

▲ 3단 합체 — MemberRepository 하나로 CRUD + 동적 쿼리 모두

```
public interface MemberRepositoryCustom { List<M> search(MemberSearchCondition c); }

@RequiredArgsConstructor // 이름 규칙 필수: 인터페이스명 + Impl
public class MemberRepositoryImpl implements MemberRepositoryCustom {
    private final JPAQueryFactory queryFactory;
    public List<M> search(MemberSearchCondition c) {
        return queryFactory.selectFrom(querydslMember)
```

```

        .leftJoin(querydslMember.team, querydslTeam)
        .where(usernameEq(c.getUsername()), ageGoe(c.getAgeGoe()), ...) // null 무시
        .fetch();
    }
}

public interface MemberRepository
    extends JpaRepository<M, Long>, MemberRepositoryCustom {} // 두 세계 통합

```

심화: CQRS Impl이 비대해지면, 화면 특화 복잡 조회는 **MemberQueryRepository** (**@Repository** + **JPAQueryFactory**만)로 분리한다. **Command**(핵심 CRUD, 느린 변경) = **JpaRepository**, **Query**(화면 조회, 잦은 변경) = 별도 클래스. 기획자가 "필터 바꿔주세요" 할 때 핵심 로직을 안 건드리고 조회 클래스만 뜯어고칠 수 있다.

⑦ 성능 최적화 — DTO 조회(Projections)와 벌크 연산

Projections: 엔티티 말고 DTO만

엔티티를 조회하면 영속성 컨텍스트가 관리(스냅샷·변경감지)하는 비용이 든다. 화면에 이름·나이만 뿌릴 거면 DTO로 필요한 필드만 뽑는다.

방식	동작	단점
Projections.bean	기본생성자 + Setter 주입	전부 문자열·순서 의존 → 런타임 에러
Projections.fields	리플렉션으로 필드 직접 주입	
Projections.constructor	생성자 순서·타입 맞춤	
@QueryProjection	DTO를 Q클래스로 → <code>new QMemberDto(...)</code>	컴파일 타임 체크(빨간 줄)

✅ **실무 No.1: @QueryProjection** DTO 생성자에 **@QueryProjection** 을 붙이면 **QMemberDto** 가 생성된다. `new QMemberDto(username, age)` 로 쓰면 순서·타입이 틀릴 때 **즉시 빨간 줄**이 떴서 앱이 터지는 걸 사전에 막는다.

페이징 최적화: PageableExecutionUtils

Count 쿼리 생략 (최적화 O)

첫 페이지인데 데이터 < size
또는 마지막 페이지 → 개수 계산 가능

fetchOne() 즉시 호출 금지

바로 실행하면 최적화 기회 상실
→ 메서드 참조(::fetchOne)로 지연

▲ COUNT(*)는 무겁다 — 필요 없을 때 자동 생략

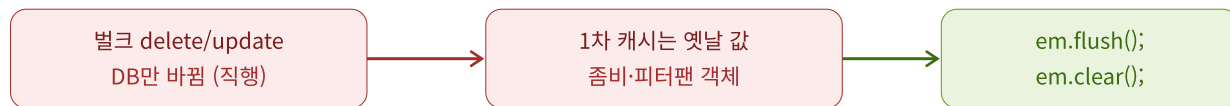
```

JPAQuery<Long> countQuery = queryFactory.select(querydslMember.count())
    .from(querydslMember).where(...); // ← left join 불필요하면 생략 = 성능↑
// 3번째 인자를 메서드 참조로! (즉시 fetchOne 하면 최적화 무력화)
return PageableExecutionUtils.getPage(content, pageable, countQuery::fetchOne);

```

벌크 연산 — 영속성 컨텍스트 불일치

벌크 연산(`update().set().execute()` / `delete().execute()`)은 영속성 컨텍스트를 무시하고 DB로 직행한다. 그래서 1차 캐시와 DB가 어긋난다.



삭제했는데 조회됨(좀비) / 나이 안 먹음(피터팬) — 1차 캐시 우선(Repeatable Read)
벌크 연산 후 반드시 캐시 초기화!

▲ 벌크 연산 후 `em.flush(); em.clear();` 는 기계적으로

⚠ **정합성 주의** 벌크 후 캐시를 안 비우면, 삭제한 데이터가 조회되는 **좀비 객체**, 나이를 안 먹는 **피터팬 객체**가 생긴다(1차 캐시 우선).
Spring Data는 `@Modifying(clearAutomatically=true)` 로 자동 처리되지만 QueryDSL 직접 사용 시엔 **수동 `em.clear()` 필수**.
소량은 Dirty Checking, 대량(수천 ↑)은 벌크.

⑧ SQL 함수 호출 — Dialect 한계 돌파

DB 고유 함수(`DATE_FORMAT` , `GROUP_CONCAT` , `REPLACE`)는 QueryDSL 메서드에 없다. `Expressions.stringTemplate` 으로 인덱스 바인딩(`{0}` , `{1}`)해서 호출한다.

```
// function() = Dialect에 등록된 함수 사용 (JPA 표준)
Expressions.stringTemplate("function('replace', {0}, {1}, {2})",
    querydslMember.username, ConstantImpl.create("김"), ConstantImpl.create("최"));

// GROUP_CONCAT: 1:N을 콤마 문자열 한 줄로 (데이터 뺏기기·N+1 방지)
.select(querydslMember.team.name,
    Expressions.stringTemplate("group_concat({0})", querydslMember.username))
.groupBy(querydslMember.team.name) // groupBy 필수
// teamA : 김철수,이영희,박민수
```

⚠ **DB 종속성** `stringTemplate` 은 DB에 종속적이다(MySQL `GROUP_CONCAT` ↔ Oracle `LISTAGG`). DB 교체 가능성이 있으면 신중히 쓰고, 필요할 때만 사용한다.

대단원 총정리 (1~5주차 로드맵)

1주차 영속성 컨텍스트(1차 캐시) → **2주차** 엔티티 매핑·생명주기 → **3주차** 연관관계·주인·프록시·N+1 → **4주차** 고급 매핑(상속·값타입·복합키) → **5주차** QueryDSL. 테이블에 끌려다니던 '코더'에서 객체 중심으로 아키텍처를 통제하는 '엔지니어'로.

5주차 한 문장 요약

QueryDSL은 쿼리를 **자바 코드로** 짜서 오타를 컴파일 타임에 잡는다. 동적 쿼리는 **Where 다중 파라미터 + BooleanExpression**으로 null을 자동 무시하며 레고처럼 조립하고, **Custom Repository(3단 합체)**로 JpaRepository와 통합한다. 조회는 **@QueryProjection**

으로 안전하게 DTO로 뽑고, 페이징은 **PageableExecutionUtils**로 count를 아끼며, 벌크 연산 뒤엔 반드시 **em.clear()**로 캐시를 비운다.

셀프 체크 (복습용)

Q1. QueryDSL이 JPQL보다 나은 결정적 이유는?

A. 자바 코드라 오타를 컴파일 시점에 잡고(런타임 장애 방지), 자동완성·타입 안전성을 지원한다.

Q2. 동적 쿼리에서 조건 메서드 반환 타입을 왜 BooleanExpression으로?

A. `.and().or()` 로 조립(Composition)이 가능하고, null 반환 시 where에서 자동 무시된다.

Q3. Custom Repository 구현체 이름 규칙은?

A. 인터페이스명 + `Impl` (예: `MemberRepositoryImpl`). Spring이 이 규칙으로 자동 연결.

Q4. DTO 조회에서 가장 안전한 방식과 이유는?

A. `@QueryProjection`. 순서·타입 오류를 컴파일 타임에 빨간 줄로 잡는다.

Q5. 벌크 연산 후 반드시 해야 할 일은?

A. `em.flush(); em.clear()`. 안 하면 좀비·피터팬 객체(1차 캐시 불일치) 발생.